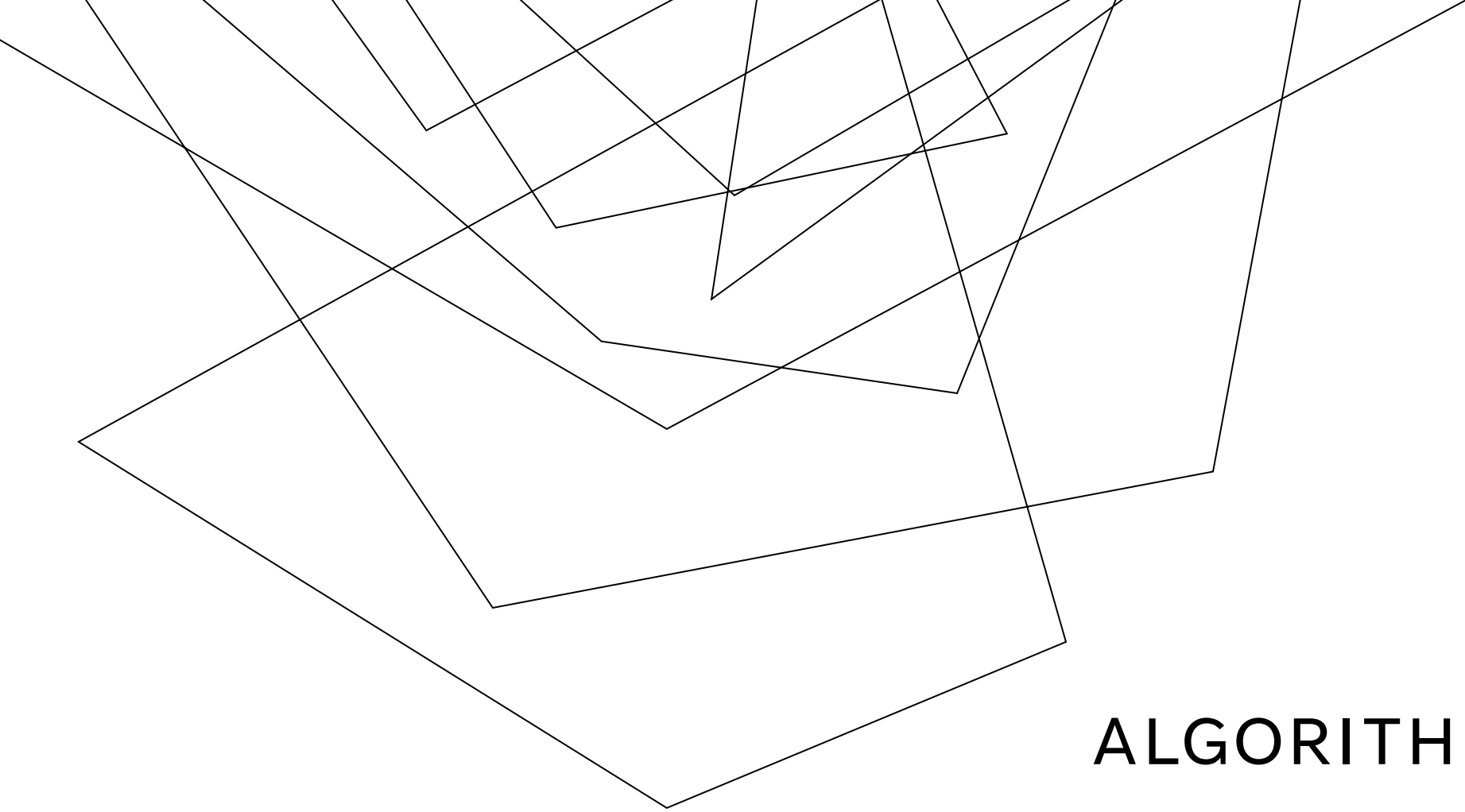


A series of thin, black, overlapping lines forming various geometric shapes like triangles and polygons, creating a complex, abstract pattern in the upper left portion of the slide.

ALGORITHMS

3. RECURRENCE





지난 시간 복습

ALGORITHM ANALYSIS

1. Algorithm analysis – Why? How?
2. Best / Worse / Average case
3. Asymptotic analysis
(Big-O, Big-Omega, Big-Theta)



ALGORITHM ANALYSIS

1. Algorithm analysis – Why? How?
2. Best / Worse / Average case
3. Asymptotic analysis
(Big-O, Big-Omega, Big-Theta)





목차

목차

1. Recurrence
2. Asymptotic analysis of recursive algorithms



An abstract graphic on a light gray background. Two thin, dark gray lines intersect. One line is nearly vertical, starting from the top center and extending towards the bottom right. The other line is nearly horizontal, starting from the top left and extending towards the middle right. The intersection point is located to the left of the word 'RECURRENCE'.

RECURRENCE

RECURRENCE

Ex) 정수 n 의 팩토리얼 ($n!$)을 계산하는 문제

- 입력 : 정수 n
- 출력 : $n!$



RECURRENCE

Ex) 정수 n 의 팩토리얼 ($n!$)을 계산하는 문제

- 입력 : 정수 n
- 출력 : $n!$

Algorithm 1

```
def factorial(n):  
    result = 1  
    for i in range(1, n+1):  
        result *= i  
    return result
```



RECURRENCE

Ex) 정수 n 의 팩토리얼 ($n!$)을 계산하는 문제

- 입력 : 정수 n
- 출력 : $n!$

Algorithm 1

```
def factorial(n):  
    result = 1  
    for i in range(1, n+1):  
        result *= i  
    return result
```

Algorithm 2

```
def factorial(n):  
    if n == 1:  
        return 1  
    return n * factorial(n-1)
```





ASYMPTOTIC ANALYSIS OF RECURSIVE ALGORITHMS

COMPLEXITY OF RECURSIVE ALGORITHMS

Ex) 정수 n 의 팩토리얼 ($n!$)을 계산하는 문제

Algorithm 1

```
def factorial(n):  
    result = 1  
    for i in range(1, n+1):  
        result *= i  
    return result
```



$$T(n) = 2n+1$$

Algorithm 2

```
def factorial(n):  
    if n == 1:  
        return 1  
    return n * factorial(n-1)
```



$$\begin{aligned} T(n) &= T(n-1) + 2 & (n > 1) \\ T(1) &= 1 & (n = 1) \end{aligned}$$



COMPLEXITY OF RECURSIVE ALGORITHMS

Ex) 정수 n 의 팩토리얼 ($n!$)을 계산하는 문제

Algorithm 1

```
def factorial(n):  
    result = 1  
    for i in range(1, n+1):  
        result *= i  
    return result
```


$$T(n) = 2n+1$$


$$O(n)$$

Algorithm 2

```
def factorial(n):  
    if n == 1:  
        return 1  
    return n * factorial(n-1)
```


$$\begin{aligned} T(n) &= T(n-1) + 2 & (n > 1) \\ T(1) &= 1 & (n = 1) \end{aligned}$$


$$??$$


METHODS OF ASYMPTOTIC ANALYSIS

1. Iterative Substitution (반복 대치)

- 더 작은 크기의 시간복잡도 함수로 반복해서 대입/치환

2. Guess & Verification (추정 후 증명)

- 결론을 추정한 후 수학적 귀납법으로 증명

3. Master Theorem (마스터 정리)

- 특정 형태의 점화식일 경우 간단히 풀 수 있음



ITERATIVE SUBSTITUTION

Ex1)

$$\begin{aligned}T(n) &= T(n-1) + n \quad (n > 1) \\T(1) &= 1\end{aligned}$$



ITERATIVE SUBSTITUTION

Ex2)

$$\begin{aligned} T(n) &= 2T(n/2) + n \quad (n > 1) \\ T(1) &= 1 \end{aligned}$$

HINT : n 을 2의 제곱수로 가정해도 됨 ($n = 2^k$)



ITERATIVE SUBSTITUTION

Ex3)

$$\begin{aligned}T(n) &= 3T(n/4) + n \quad (n > 1) \\T(1) &= 1\end{aligned}$$



GUESS AND VERIFICATION

Ex2)

$$\begin{aligned} T(n) &= 2T(n/2) + n \quad (n > 1) \\ T(1) &= 1 \end{aligned}$$

- **Guess** : $T(n) = O(n \log n)$, 즉, $T(n) \leq c * n \log n$



GUESS AND VERIFICATION

Ex2)

$$\begin{aligned} T(n) &= 2T(n/2) + n \quad (n > 1) \\ T(1) &= 1 \end{aligned}$$

- **Guess** : $T(n) = O(n \log n)$, 즉, $T(n) \leq c * n \log n$

- **Verification** : 수학적 귀납법을 통해 증명

즉,

$T\left(\frac{n}{2}\right) \leq c * \frac{n}{2} \log \frac{n}{2}$ 이 성립한다고 가정할 때

$T(n) \leq c * n \log n$ 이 성립함을 보이면 됨

cf) Base case 는 일반적으로 증명하지 않아도 됨



GUESS AND VERIFICATION

Ex2)

$$\begin{aligned} T(n) &= 2T(n/2) + n \quad (n > 1) \\ T(1) &= 1 \end{aligned}$$

- **Guess** : $T(n) = O(n \log n)$, 즉, $T(n) \leq c * n \log n$
- **Verification** :



GUESS AND VERIFICATION

Ex4)

$$T(n) = 2T(n/2) + 1$$

- **Guess :** $T(n) \leq cn$



GUESS AND VERIFICATION

Ex4)

$$T(n) = 2T(n/2) + 1$$

- **Guess :** $T(n) \leq cn$
- **Guess2 :** $T(n) \leq cn - 2$



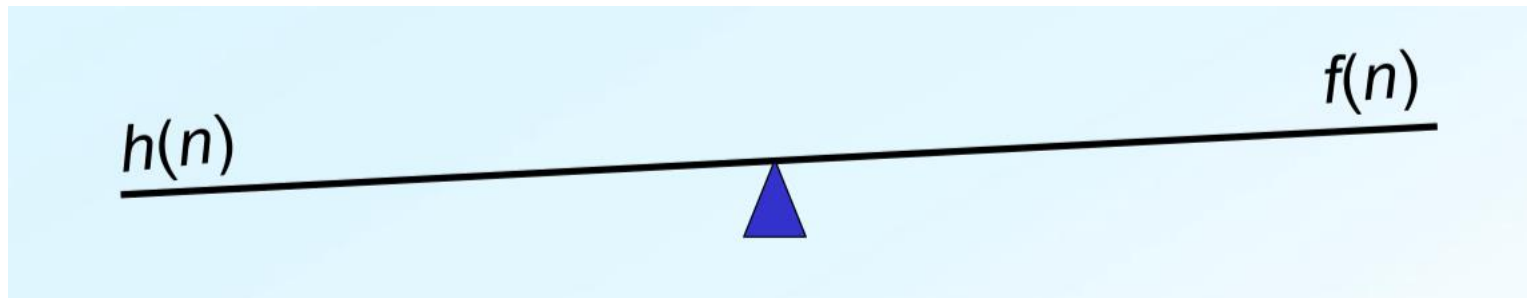
MASTER THEOREM

다음 형태의 점화식에 대해서

$$T(n) = aT(n/b) + f(n) \quad (n > 1)$$

$h(n) = n^{\log_b a}$ 로 두고 $f(n)$ 과 비교

→ $h(n)$, $f(n)$ 중 더 무거운 term이 $T(n)$ 의 전체 시간복잡도를 결정



MASTER THEOREM

다음 형태의 점화식에 대해서

$$T(n) = aT(n/b) + f(n) \quad (n > 1),$$

$$h(n) = n^{\log_b a}$$

1. $h(n)$ 이 더 큰 경우
→ $T(n) = \Theta(h(n))$
2. $f(n)$ 이 더 큰 경우
→ $T(n) = \Theta(f(n))$
3. $h(n), f(n)$ 이 동률인 경우
→ $T(n) = \Theta(h(n)\log n)$



MASTER THEOREM

다음 형태의 점화식에 대해서

$$T(n) = aT(n/b) + f(n) \quad (n > 1),$$

$$h(n) = n^{\log_b a}$$

- Iterative substitution을 통한 간단한 유도



MASTER THEOREM

Ex5)

$$T(n) = 2T(n/4) + n$$

Ex6)

$$T(n) = 2T(n/2) + n$$



Two thin, dark gray lines intersect diagonally on a light gray background. One line runs from the top-left towards the bottom-right, and the other runs from the top-right towards the bottom-left. They cross each other in the upper-left quadrant of the image.

SUMMARY

SUMMARY

1. Recurrence

2. Asymptotic analysis of recursive algorithms

- Iterative substitution
- Guess and verification
- Master theorem



NEXT LECTURE

Sorting (정렬)



Two thin, dark gray lines intersect diagonally on a light gray background. One line runs from the top-left towards the bottom-right, and the other runs from the top-right towards the bottom-left. They cross each other in the upper-left quadrant of the image.

QUESTIONS?